# Assignment-3

**HLCV**

**Saarland University – Summer Semester 2025**

**Shreyansh Tripathi (7056894)**
**Ashraf Hesham Raghey Raghey Embaby (7076816)**
shtr00001@stud.uni-saarland.de

## 1 Exercise 1

In the first part of the question one, we have implemented initialized the model parameters from the config file and created a model with 5 convolutional blocks as described the problem. We then trained this model on the cifar 10 dataset for 10 epochs and fetched the training and eval loss in each epoch. We observed from each epoch that the training loss has gradually lowered indicating that the model was fitting to the training data. We observed a decrease in the eval loss as well until epoch 9 where there was a slight spike in the loss indicating the model must be overfitting on the data.

We can also visualize the filters for the first layer before and after training the model in 1. There is a difference in the gradient of each filter after training the model indicating the backpropogation was at work and that the model was learning the parameters. The filter before training looks like a random initialization while filter after training has learn low level features like edges, corner etc.

In the CIFAR10_CNN.log file we also are provided with additional metric values such as Top 1 and Top 5 accuracies. Both the accuracies increase in each epoch, resulting in high eval_top1 and eval_top5 in the 10th epoch.

Next, we also evaluate the model on 10,000 unseen images to which the considerably lower eval loss of .65 indicates that the model did learn how to generalize. The top1 and top5 accuracies as well are considerable.

In the second part of the question one, we implemented the code which gives us the number of trainable parameters for each layer and the total number of trainable parameters. Trainable parameters are weights and biases that are obtained from the convolutional layers and not the max-pool layers. The number of weights increases as the layer size increases and number of biases is equal to the number of filters in each layer. As the total number of trainable parameters is 7.5M +, the model is less complex in comparison to AlexNet which has 60M trainable parameters.

## 2 Exercise 2

### 2.1 Exercise 2.1

Figure 2 compares the loss curves, top1 and top5 accuracy curves with and without batch normalization. As we can see, the batch normalization consistently outperforms the vanilla version in all metrics in evaluation settings, despite achieving comparable performance on the training set, implying that it helps improve generalization.

BN achieves around 0.85 in the top-1 matrix in evaluation in comparison to 0.81 with the vanilla version, even though both achieve the same results on the training set. Similar results can be observed in the top-5 metric with a narrower gap and in the loss with BN achieving 0.6, which is almost 0.2

Preprint. Under review.

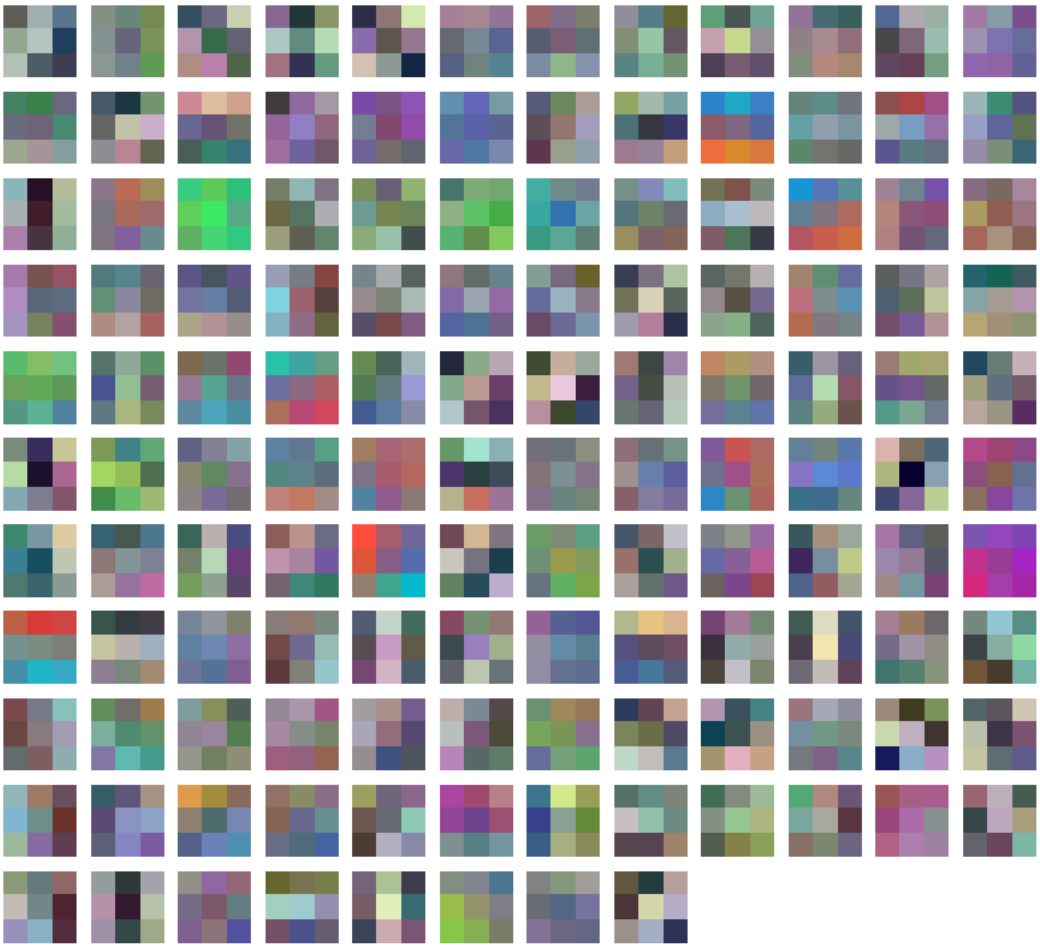


Figure 1: Visualization of the layer 1 filter weights after training

percentage points lesser than the vanilla model.

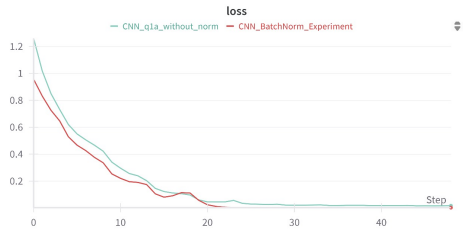
Another interesting point is that this generalization happens rather quickly with accuracies and losses stabalizing at around epoch number 20.

### 2.2 Exercise 2.2

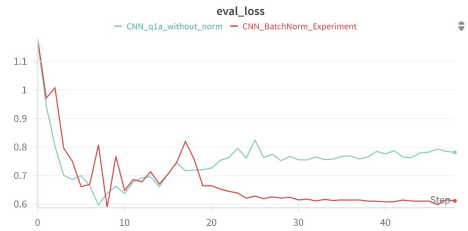
We compared the best model and latest model on the training and the validation set. Following observations were made:

Although training was done for 50 epochs, the best model we got was at epoch 34 with the following stats:

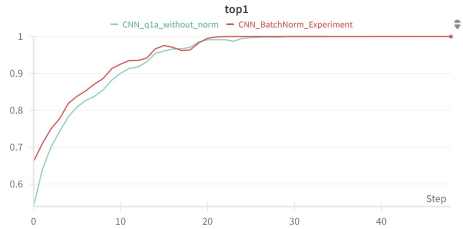
- Training Accuracy Top1: 1.0
- Training Accuracy Top5: 1.0
- Val Accuracy Top1: 0.8492
- Val Accuracy Top5: 0.98
- Training Loss: 0.0025



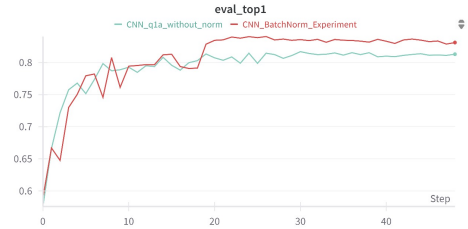

(a) Training Loss



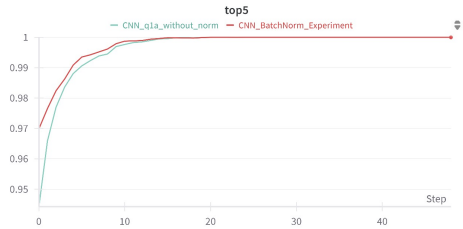

(b) Validation Loss



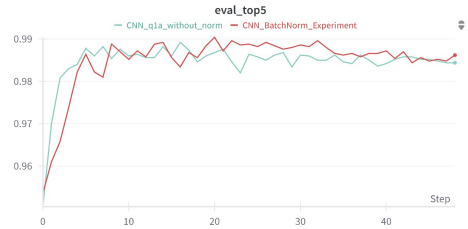

(c) Top1 Training Accuracy




(d) Top1 Validation Accuracy




(e) Top 5 Training Accuracy




(f) Top 5 Validation Accuracy

Figure 2: Comparison of Metrics between models with and without Batch Normalization

- Val Loss: 0.61

Similarly for the latest model the stats are:

- Training Accuracy Top1: 1.0
- Training Accuracy Top5: 1.0
- Val Accuracy Top1: 0.8311
- Val Accuracy Top5: 0.98
- Training Loss: 0.0023
- Val Loss: 0.58

It is evident from the above comparison that the model starts to lose performance after 34th epoch albeit very less, and that the best and the final models aren't the same level of performance. This means that the model overfits and thus early stopping can happen sometime after step 34.

### 2.3 Exercise 2.3

Figure 3 shows that Early stop significantly reduced the number of to 38 as the improvement of top1 metric on validation set stopped improving afterwards. This means that the early stop helps in stoping the training ealier and helps in saving resources and cost.

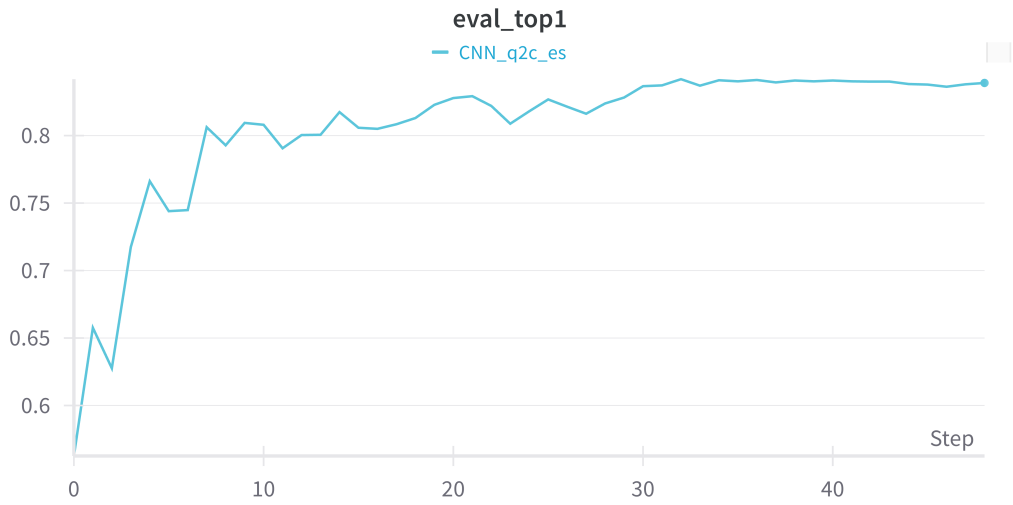



Figure 3: Top1 Validation accuracy of the Early stopping experiment with number of steps = 4

# 3 Exercise 3

## 3.1 Exercise 3.1

Figure 4 show the performance of the model trained using different augmented datasets. The augmentations we used are:

- Horizontal flip with probability p = 0.5
- Color inversions with probability p = 0.5
- Grayscaling with p = 0.5
- Random Rotation with degree d = 60 in a random direction
- Gaussian blur with kernel size = 11
- Random Contrast

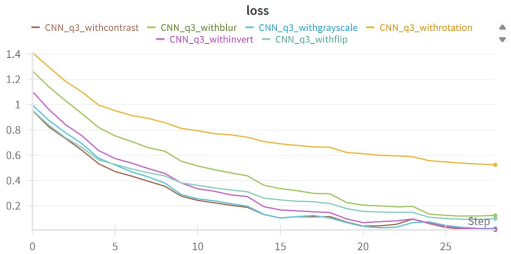
We found that horizontal flipping gives the best evaluation performance across all metrics with Top-1 = .83 and loss = 0.54, and that Gaussian blur achieves the poorest results.

A curious thing is that all of the augmentations individually performed slightly worse than the BatchNorm model used in 2.1. This may be because the models with augmentation may need more time and different hyperparameters to train better. Additionally, the normalization arguments may be different.

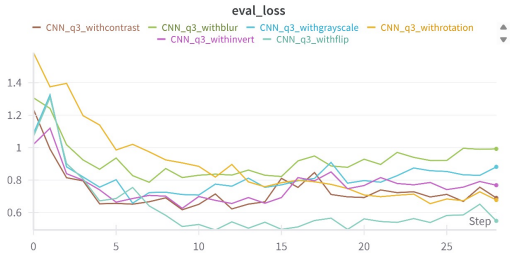
We used the top three performing augmentations: Rotation, Flipping, and Contrast to build a final model which performed better than all the other individual methods except for Flipping. The results can be seen in 5. This may be because more augmentation would require more epochs to train. So we trained the model for 60 epochs and got an top1 accuracy of 83.35 outperforming models with singular augmentations.

## 3.2 Exercise 3.2

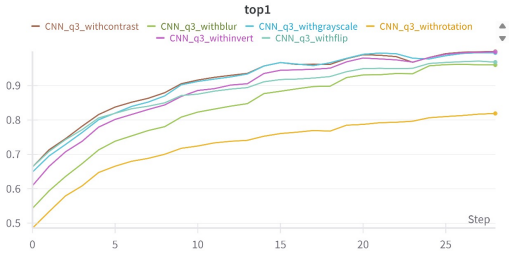
Figure 6 plot the training and validation accuracies for different values of dropout (0.1 - 0.9). As we can see, we get the best results with p = 0.1 and p = 0.35, which both achieve similar results across all metrics, with p = 0.6 a little bit worse but with no signs of over- or underfitting. However, with p = 0.9, the model achieves very poor results across all metrics, even in the training settings, implying that the constraints on the model are not flexible enough for the model to train, causing the model to underfit.



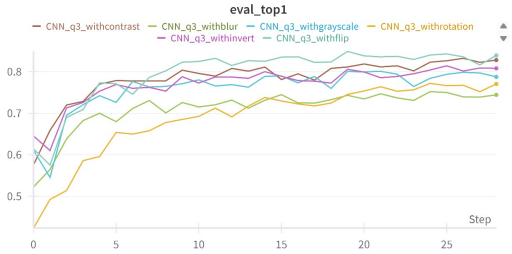

(a) Training Loss



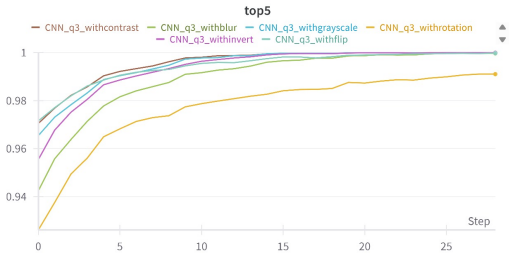

(b) Validation Loss



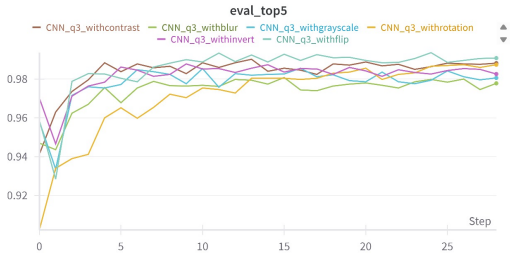

(c) Top1 Training Accuracy




(d) Top1 Validation Accuracy




(e) Top 5 Training Accuracy




(f) Top 5 Validation Accuracy

Figure 4: Comparison of Metrics between models with different augmentations. These Augmentations are applied in isolation to make them comparable.

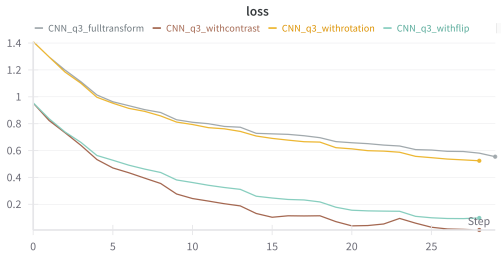
Also, the models seem to attain goof accuracies even at lower epochs (between 15 and 20 in this case), but continue to improve very slowly.

## 4 Exercise 4:

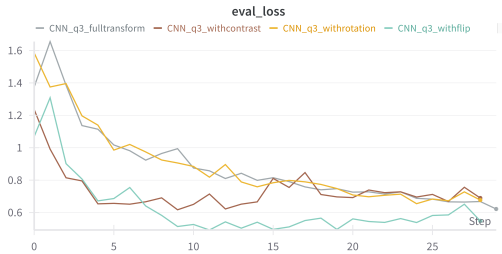
Figure 7 illustrates the effects of using a pretrained network as a base for training the model under three settings:

1. Freeze: freezing the pretrained weights and training only newly added layers on top of the network
2. Full: training the whole model using the pretrained network weights as initializations.
3. None: training the whole model from scratch (Baseline)

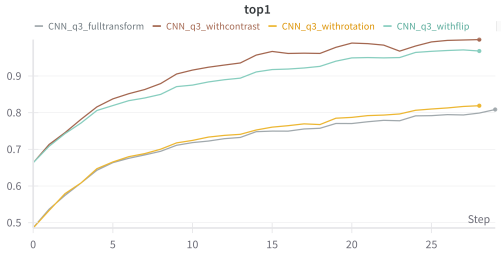
As we can see, freezing the pretrained weights doesn't perform well on either training or evaluation settings, which can be attributed to the domain shift between the ImageNet dataset and the CIFAR-10 dataset. Meanwhile, using the pretrained weights as initializations yields the best performance in all metrics, and it shows faster convergence than the baseline model.



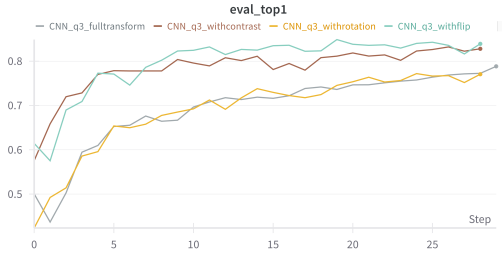

(a) Training Loss



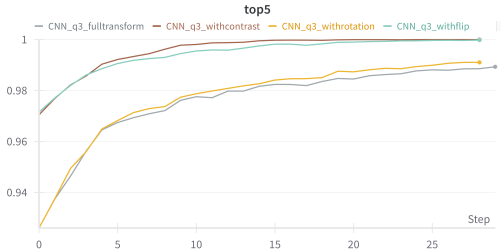

(b) Validation Loss



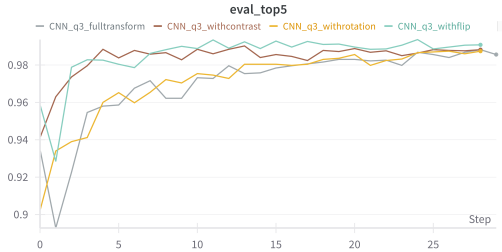

(c) Top1 Training Accuracy




(d) Top1 Validation Accuracy




(e) Top 5 Training Accuracy




(f) Top 5 Validation Accuracy

Figure 5: Comparison of Metrics between models with different augmentations combinations. The final version has top 3 augmentation from previous experiment.

## Resources

To access WandB dashboard please visit: `https://wandb.ai/shreyanshtripathi03-universit-t-des-saarlandes-saarland-/HLCV`

## Acknowledgements

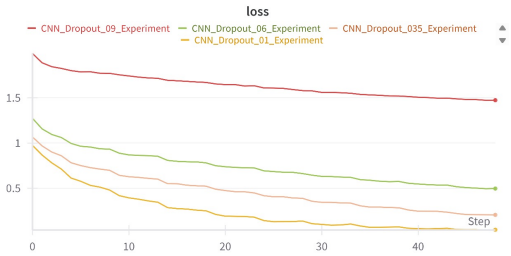
For the implementation exercises, python3, pytorch [1] is used.

ChatGPT [2]`https://chatgpt.com/` and stackoverflow `https://stackoverflow.com/` are used for refining latex code.

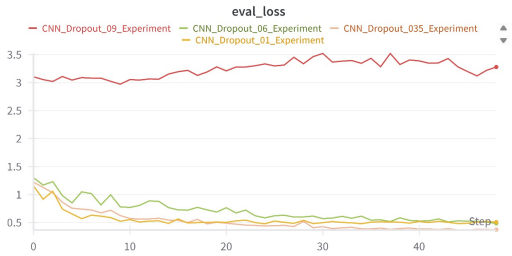
## References

[1] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

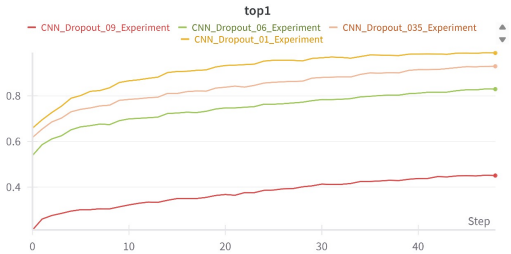
[2] OpenAI. Chatgpt: Personal communication, November 2024. Accessed on November 18, 2024. This is an AI language model and does not provide retrievable sources.



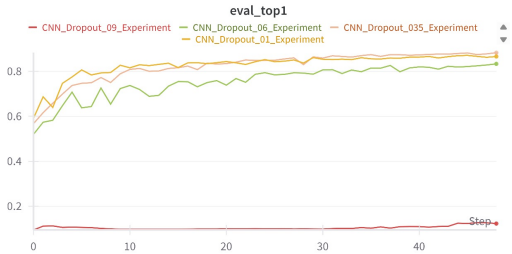

(a) Training Loss



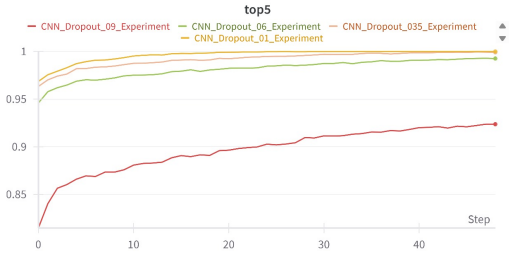

(b) Validation Loss



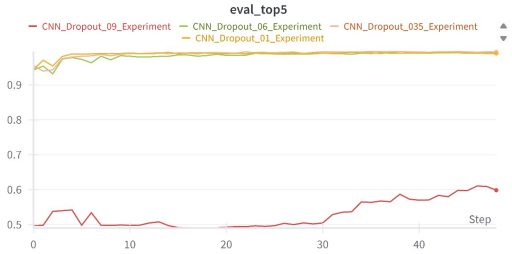

(c) Top1 Training Accuracy




(d) Top1 Validation Accuracy



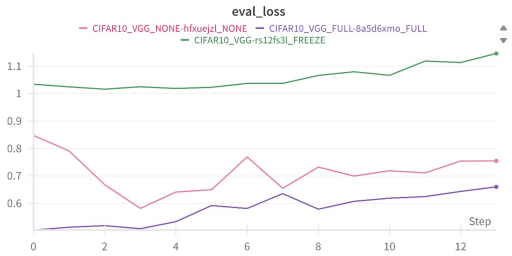

(e) Top 5 Training Accuracy



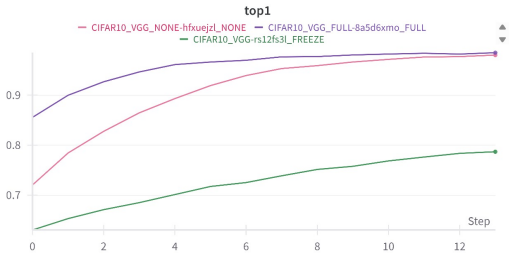

(f) Top 5 Validation Accuracy

Figure 6: Comparison of Metrics between models with different dropout rates.

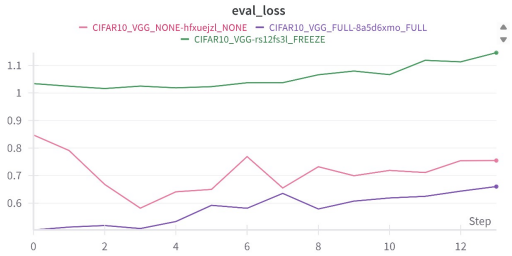
(a) Training Loss



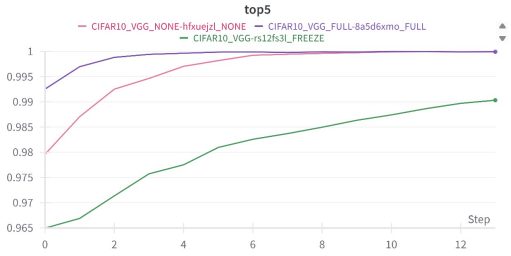

(b) Validation Loss



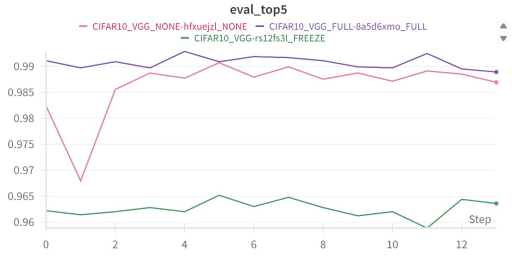

(c) Top1 Training Accuracy




(d) Top1 Validation Accuracy




(e) Top 5 Training Accuracy




(f) Top 5 Validation Accuracy

Figure 7: Comparison of Metrics between models with different VGG variants trained